



p7g1/ Prof. Carlos Costa

Base de Dados para Gestão de Eventos

José Silva (103248), Joana Silva (119844)

Índice

Índice.....	2
Introdução.....	3
Análise de Requisitos.....	4
Entidades.....	6
DER.....	7
ER.....	8
DDL.....	9
DML.....	9
Stored Procedure.....	10
View.....	12
UDF.....	13
Triggers.....	14
Índices.....	16
Cursor.....	17
Conclusão.....	18

Introdução

No âmbito da unidade curricular de Base de Dados, da Licenciatura em Engenharia de Computadores e Informática, no ano letivo 2025/2026, foi-nos proposto a criação de um projeto que fizesse a gestão de um sistema funcional e com complexidade razoável aplicável ao mundo real, tendo como objetivo a criação e desenvolvimento de um sistema de bases de dados relacional para suportar uma plataforma centralizada de gestão de eventos. O seguinte relatório irá descrever sucintamente o nosso projeto dando ênfase nas partes que achamos mais fulcrais para o funcionamento do mesmo.

Em anexo a este relatório será enviado o código contém os scripts usado para criar a base de dados e utilizar a mesma e um WFA em C# com o seu código.

Ao longo do relatório, mostraremos como foi implementada a base de dados, recorrendo ao SQL Server Management Studio e guardando as queries executadas no servidor do IETTA. Para que seja possível a testagem da nossa aplicação noutra base de dados para além daquela que foi usada no desenvolvimento nas aulas práticas é necessário aceder ao ficheiro Program.cs e mudar os atributos BD_STRING, username e password para as credenciais desejadas.

```
1 using System;
2 using System.Windows.Forms;
3
4 namespace FormsProjeto
5 {
6     20 references
7     internal static class Program
8     {
9         public static string DbUser = "p7g1";
10        public static string DbPassword = "-1683641040@BD";
11
12        public static string ConnectionString = $"Server=192.168.182.10;Database=p7g1;User Id={DbUser};Password={DbPassword};TrustServerCertificate=True;";
13
14        [STAThread]
15        0 references
16        static void Main()
17        {
18            Application.EnableVisualStyles();
19            Application.SetCompatibleTextRenderingDefault(false);
20
21            Application.Run(new Form1());
22        }
23    }
24 }
```

Segue também um vídeo demonstrativo usado para a apresentação.

NOTA: Para ver melhor as imagens, tanto do código, quanto dos esquemas, consultar os ficheiros referentes aos mesmos.

Análise de Requisitos

De modo a garantir o bom funcionamento do sistema, foram definidos um conjunto de requisitos que considerámos essenciais para o sistema, sendo eles divididos em duas categorias. Foram também estabelecidas algumas regras de negócio, estas foram importantes pois permitiram o desenvolvimento de restrições a aplicar ao nosso sistema.

Requisitos funcionais:

1. RF01 - Gestão de Clientes: O sistema deve permitir registar, consultar, atualizar e remover dados de clientes (nome, contactos, NIF).
2. RF02 - Gestão de Tipos de Evento: O sistema deve ter um catálogo de tipos de eventos possíveis (ex: Casamento, Batizado, Festa de Aniversário, Funeral, Conferência Corporativa).
3. RF03 - Gestão de Locais: O sistema deve manter um catálogo dos espaços disponíveis para aluguer, incluindo a sua morada, capacidade máxima de pessoas e o custo base de aluguer.
4. RF04 - Gestão de Atividades Extra: O sistema deve permitir gerir os serviços complementares (ex: catering, música, decoração, fotografia), definindo o seu custo e descrição.
5. RF05 - Marcação de Eventos: O sistema deve permitir a criação de um evento, associando-o a um cliente, a um local e definindo a data, hora e o número de convidados previstos.
6. RF06 - Associação de Atividades a Eventos: O sistema deve permitir adicionar ou remover múltiplas atividades extra a um evento já agendado.

7. RF07 - Cálculo de Custos (Faturação base): O sistema deve ser capaz de calcular e apresentar o valor total do evento, somando o custo de aluguer do local com o custo das atividades extra contratadas.
8. RF08 - Associação de Atividades a Tipos de Evento: O sistema deve permitir definir que atividades extra estão disponíveis para cada tipo de evento.

Requisitos não funcionais:

1. RNF01 - Sistema Gestor de Base de Dados: O armazenamento e a gestão dos dados serão realizados exclusivamente no Microsoft SQL Server.
2. RNF02 - Interface Gráfica: A camada de apresentação será desenvolvida em C# utilizando a tecnologia Windows Forms (Visual Studio).
3. RNF03 - Comunicação de Dados: A interação entre a interface e a base de dados será feita obrigatoriamente e exclusivamente em linguagem SQL nativa.

Requisitos de negócio:

1. RN01 - Prevenção de Overbooking: Um mesmo local não pode ter dois eventos agendados para a mesma data e bloco de horas.
2. RN02 - Limite de Lotação: O número de convidados de um evento não pode ultrapassar a capacidade máxima definida para o local escolhido.
3. RN03 - Integridade de Datas: Não é possível agendar eventos para datas passadas no momento da marcação.
4. RN04 - Exclusividade de Atividades: Num mesmo evento, a mesma atividade extra não pode ser adicionada em duplicado.
5. RN05 - Restrição de Atividades por Tipo de Evento: Apenas é permitido adicionar a um evento atividades extra que estejam previamente aprovadas/associadas ao tipo desse mesmo evento.

Entidades

Evento:

É a entidade central do sistema. Regista-se a marcação de um evento, associando uma data, hora, duração e número de participantes a um determinado cliente, local e tipo de evento. É identificada pelo ID_evento.

Cliente:

Representa as pessoas que contratam os serviços de gestão. É identificada pelo NIF.

Local:

Representa os espaços físicos disponíveis para a realização dos eventos (ex: salões, auditórios). Possui atributos como capacidade máxima e preço base, sendo identificada por ID_local.

Tipo_Evento:

Funciona como um catálogo dos tipos de eventos suportados pelo sistema (ex: Casamento, Conferência, Batizado). É identificada por ID_tipo.

Atividade:

Representa qualquer serviço extra ou opcional que pode ser adicionado a um evento para além do aluguer do local. É identificada por ID_atividade.

DER

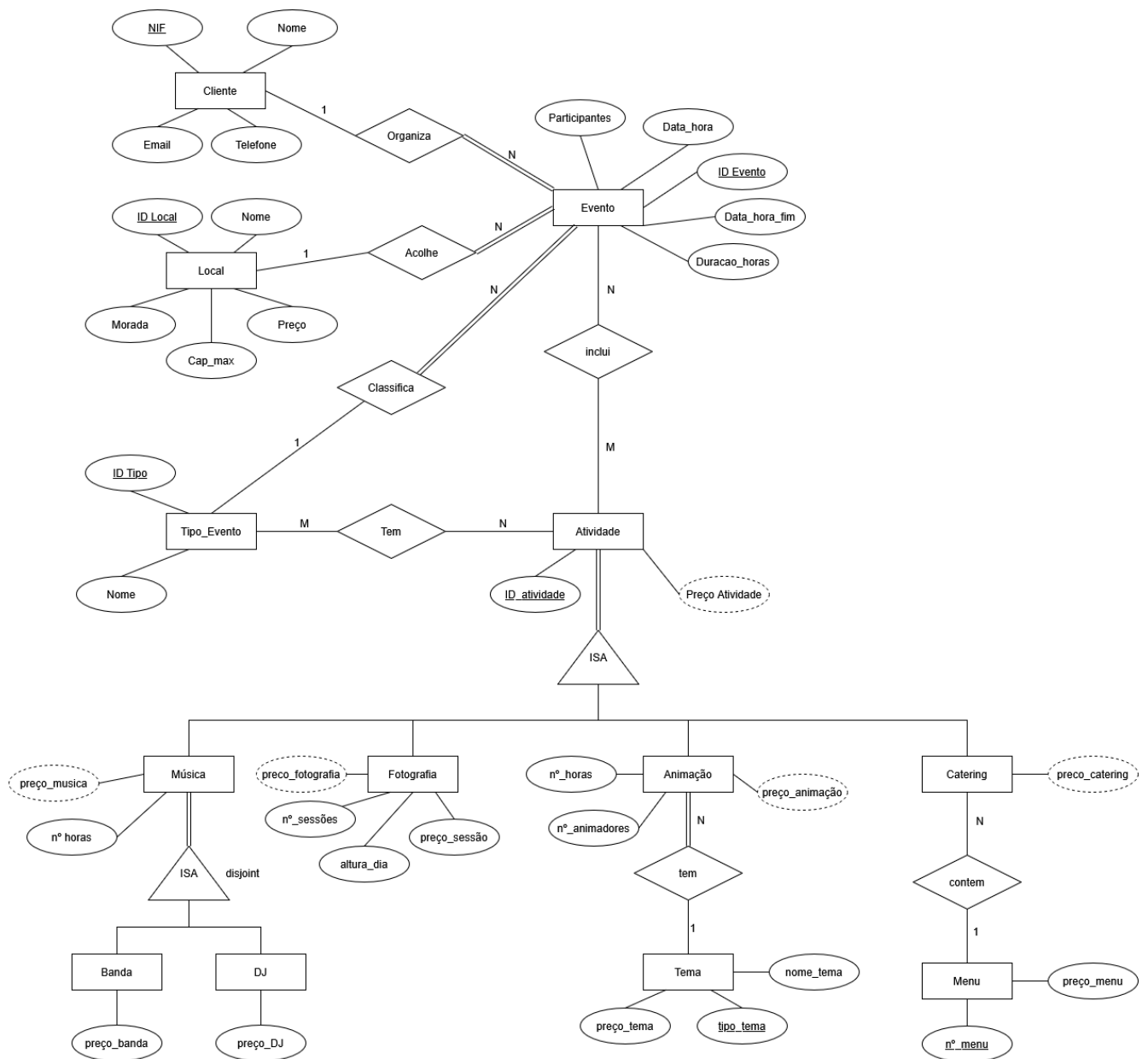


Figura1: Diagrama Entidade-Relacionamento

ER

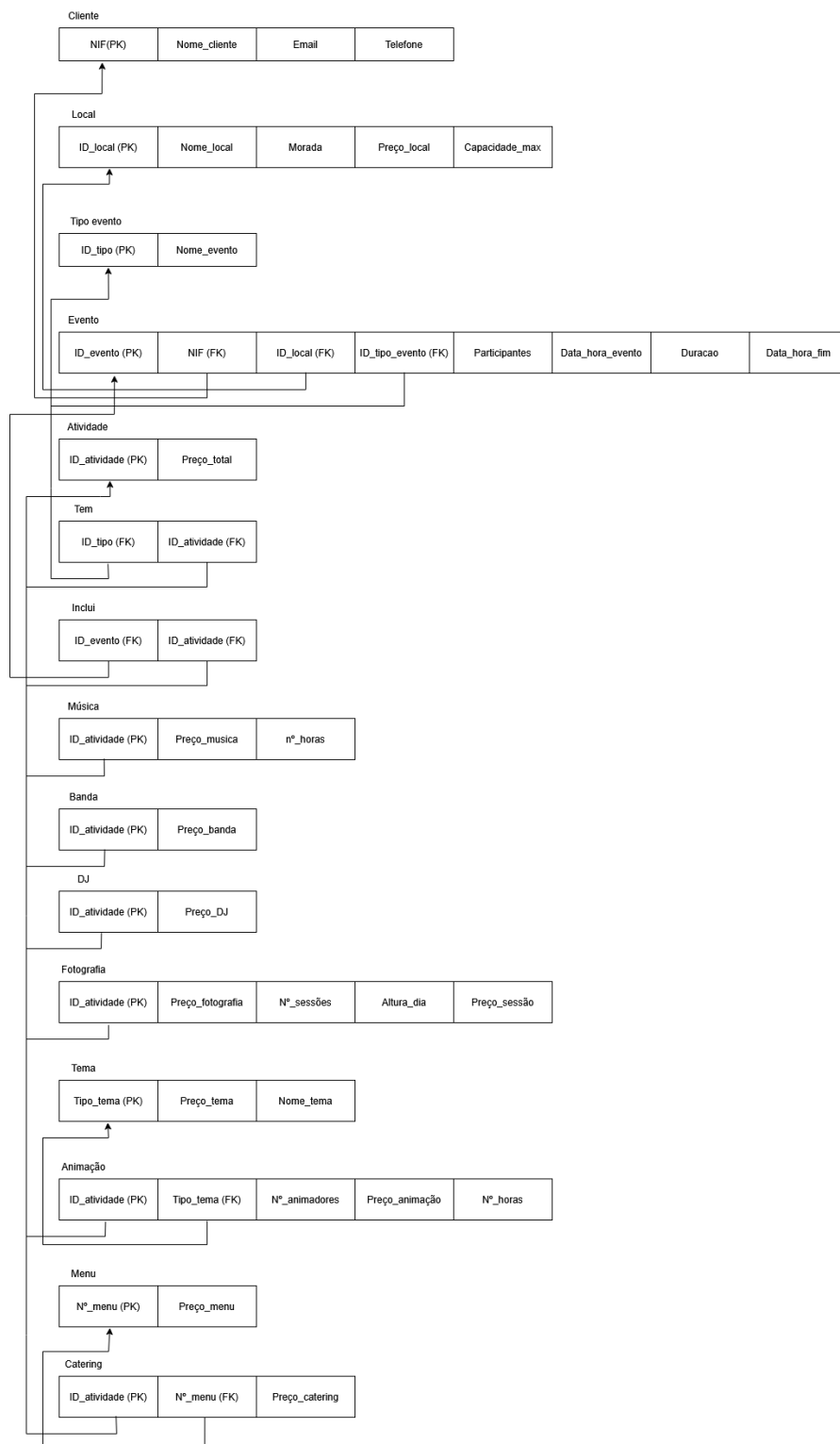


Figura 2: Esquema relacional

DDL

O código DDL (*Data Definition Language*) foi fundamental na criação das tabelas idealizadas para a base de dados. Permitiu-nos definir a estrutura das tabelas e interligações entre as mesmas. Devido ao seu tamanho não poderá ser aqui integralmente incluído, sendo enviado em anexo, no entanto poderemos incluir um exemplo de criação de uma das tabelas.

```
***** Object: Table [PROJETO_GESTAO_EVENTOS].[Banda]    Script Date: 03/06/2026 02:35:30 *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [PROJETO_GESTAO_EVENTOS].[Banda](
    [preco_banda] [decimal](10, 2) NOT NULL,
    [ID_atividade_banda] [int] NOT NULL,
    PRIMARY KEY CLUSTERED
    (
        [ID_atividade_banda] ASC
    )
    WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
) ON [PRIMARY]
GO
```

Figura 3: Código DDL

DML

O código DML (*Data Management Language*) foi o que nos permitiu inserir dados concretos nas tabelas criadas, respeitando o tipo de dados escolhido para cada coluna. Tal como o código DDL, este também tem um grande número de linhas, pelo que apenas iremos incluir um exemplo para algumas tabelas.

```
INSERT [PROJETO_GESTAO_EVENTOS].[Cliente] ([NIF], [Nome_cliente], [Email], [Telefone]) VALUES (N'100000001', N'Ana Costa', N'ana@empresa.com', N'910000001')
INSERT [PROJETO_GESTAO_EVENTOS].[Cliente] ([NIF], [Nome_cliente], [Email], [Telefone]) VALUES (N'100000002', N'Bruno Silva', N'bruno@gmail.com', N'920000002')
INSERT [PROJETO_GESTAO_EVENTOS].[Cliente] ([NIF], [Nome_cliente], [Email], [Telefone]) VALUES (N'100000003', N'Carla Dias', N'carla@hotmail.com', N'930000003')
INSERT [PROJETO_GESTAO_EVENTOS].[Cliente] ([NIF], [Nome_cliente], [Email], [Telefone]) VALUES (N'100000004', N'Diogo Santos', N'diogo@outlook.com', N'940000004')
INSERT [PROJETO_GESTAO_EVENTOS].[Cliente] ([NIF], [Nome_cliente], [Email], [Telefone]) VALUES (N'100000005', N'Elsa Rocha', N'elsa@empresa.pt', N'950000005')
INSERT [PROJETO_GESTAO_EVENTOS].[Cliente] ([NIF], [Nome_cliente], [Email], [Telefone]) VALUES (N'200000001', N'Mariana Marques', N'marianamarques20@gmail.com', N'123456789')
INSERT [PROJETO_GESTAO_EVENTOS].[Cliente] ([NIF], [Nome_cliente], [Email], [Telefone]) VALUES (N'234567888', N'Pedro', N'pe@mail.com', N'963741852')
INSERT [PROJETO_GESTAO_EVENTOS].[Cliente] ([NIF], [Nome_cliente], [Email], [Telefone]) VALUES (N'235689741', N'Maria José', N'mj@fake.pt', N'968574321')
INSERT [PROJETO_GESTAO_EVENTOS].[Cliente] ([NIF], [Nome_cliente], [Email], [Telefone]) VALUES (N'235689748', N'Pedro Manuel', N'pme@fake.pt', N'976431852')
INSERT [PROJETO_GESTAO_EVENTOS].[Cliente] ([NIF], [Nome_cliente], [Email], [Telefone]) VALUES (N'266899833', N'Maria Mabel', N'mm@wdsjs.com', N'965274789')
INSERT [PROJETO_GESTAO_EVENTOS].[Cliente] ([NIF], [Nome_cliente], [Email], [Telefone]) VALUES (N'999999999', N'José', N'fsdjsd@gmail.com', N'999999999')
GO
SET IDENTITY_INSERT [PROJETO_GESTAO_EVENTOS].[Local] ON
INSERT [PROJETO_GESTAO_EVENTOS].[Local] ([ID_local], [Nome_local], [Morada], [Max_capacidade], [Preco_local]) VALUES (1, N'Auditório Principal', N'Rua de Aveiro, 10', 200, CAST(1000.00 AS Decimal(10, 2)))
INSERT [PROJETO_GESTAO_EVENTOS].[Local] ([ID_local], [Nome_local], [Morada], [Max_capacidade], [Preco_local]) VALUES (2, N'Jardim de Inverno', N'Av. Central, 45', 500, CAST(1500.00 AS Decimal(10, 2)))
INSERT [PROJETO_GESTAO_EVENTOS].[Local] ([ID_local], [Nome_local], [Morada], [Max_capacidade], [Preco_local]) VALUES (3, N'Salão de Festas', N'Praça da República, 5', 150, CAST(600.00 AS Decimal(10, 2)))
INSERT [PROJETO_GESTAO_EVENTOS].[Local] ([ID_local], [Nome_local], [Morada], [Max_capacidade], [Preco_local]) VALUES (4, N'Quinta do Mario', N'Rua do Armário, 32', 500, CAST(2000.00 AS Decimal(10, 2)))
INSERT [PROJETO_GESTAO_EVENTOS].[Local] ([ID_local], [Nome_local], [Morada], [Max_capacidade], [Preco_local]) VALUES (5, N'Salão Pedro Mário', N'Rua da Estrada, 34', 1500, CAST(4500.00 AS Decimal(10, 2)))
INSERT [PROJETO_GESTAO_EVENTOS].[Local] ([ID_local], [Nome_local], [Morada], [Max_capacidade], [Preco_local]) VALUES (6, N'Auditório Pedro Rui', N'Rua da Esquina, 43', 500, CAST(1500.00 AS Decimal(10, 2)))
INSERT [PROJETO_GESTAO_EVENTOS].[Local] ([ID_local], [Nome_local], [Morada], [Max_capacidade], [Preco_local]) VALUES (7, N'Salão Grande', N'Rua Perto, 65', 700, CAST(2000.00 AS Decimal(10, 2)))
INSERT [PROJETO_GESTAO_EVENTOS].[Local] ([ID_local], [Nome_local], [Morada], [Max_capacidade], [Preco_local]) VALUES (8, N'Quinta da Natureza', N'Rua do Mato, 43', 800, CAST(2500.00 AS Decimal(10, 2)))
SET IDENTITY_INSERT [PROJETO_GESTAO_EVENTOS].[Local] OFF
GO
SET IDENTITY_INSERT [PROJETO_GESTAO_EVENTOS].[Tipo_evento] ON
INSERT [PROJETO_GESTAO_EVENTOS].[Tipo_evento] ([ID_tipo], [Nome_tipo_evento]) VALUES (1, N'Conferência')
INSERT [PROJETO_GESTAO_EVENTOS].[Tipo_evento] ([ID_tipo], [Nome_tipo_evento]) VALUES (2, N'Casamento')
INSERT [PROJETO_GESTAO_EVENTOS].[Tipo_evento] ([ID_tipo], [Nome_tipo_evento]) VALUES (3, N'Aniversário')
INSERT [PROJETO_GESTAO_EVENTOS].[Tipo_evento] ([ID_tipo], [Nome_tipo_evento]) VALUES (4, N'Gala Corporativa')
INSERT [PROJETO_GESTAO_EVENTOS].[Tipo_evento] ([ID_tipo], [Nome_tipo_evento]) VALUES (5, N'Batizado')
INSERT [PROJETO_GESTAO_EVENTOS].[Tipo_evento] ([ID_tipo], [Nome_tipo_evento]) VALUES (6, N'Festival')
INSERT [PROJETO_GESTAO_EVENTOS].[Tipo_evento] ([ID_tipo], [Nome_tipo_evento]) VALUES (8, N'Gala')
INSERT [PROJETO_GESTAO_EVENTOS].[Tipo_evento] ([ID_tipo], [Nome_tipo_evento]) VALUES (9, N'Apresentação')
SET IDENTITY_INSERT [PROJETO_GESTAO_EVENTOS].[Tipo_evento] OFF
GO
```

Figura 4: Código DML

Stored Procedure

Para garantir a coesão entre os preços das atividades dentro de cada uma das suas categorias e o preço final de cada atividade foi criado um stored procedure que irá manter os valores corretos.

```
USE [p7g1]
GO
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
ALTER PROCEDURE [PROJETO_GESTAO_EVENTOS].[sp_SincronizarPrecosAtividades]
AS
BEGIN
    UPDATE A
    SET A.Preco_atividade = V.PrecoFinal
    FROM PROJETO_GESTAO_EVENTOS.Atividade A
    INNER JOIN PROJETO_GESTAO_EVENTOS.vw_PrecosAtividades V ON A.ID_atividade = V.ID_atividade;
END;
```

Figura 5: Código da Stored Procedure que sincroniza o preço das atividades.

Para a finalização do sistema de reservas foi ainda criado outro stored procedure, mostrado na página seguinte, que irá gerar a fatura referente ao evento marcado.

```

SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
ALTER PROCEDURE [PROJETO_GESTAO_EVENTOS].[sp_GerarFaturaEvento]
    @ID_Evento INT
AS
BEGIN
    DECLARE @Data_evento DATETIME;
    DECLARE @NIF_Cliente VARCHAR(9);
    DECLARE @Nome_Local VARCHAR(20);
    DECLARE @Custo_Local DECIMAL(10,2);
    DECLARE @Custo_Total_Calculado DECIMAL(10,2);

    DECLARE @ID_atividade INT;
    DECLARE @Preco_Atividade DECIMAL(10,2);

    SELECT
        @Data_evento = E.Data_hora_evento,
        @NIF_Cliente = E.Cliente_evento,
        @Nome_Local = L.nome_local,
        @Custo_Local = L.preco_local
    FROM PROJETO_GESTAO_EVENTOS.Evento E
    INNER JOIN PROJETO_GESTAO_EVENTOS.[Local] L ON E.Local_evento = L.ID_local
    WHERE E.ID_evento = @ID_Evento;

    IF @Data_evento IS NULL
    BEGIN
        RAISERROR('Evento não encontrado.', 16, 1);
        RETURN;
    END

    SET @Custo_Total_Calculado = PROJETO_GESTAO_EVENTOS.fn_CalcularCustoTotalEvento(@ID_Evento);

    PRINT '=====';
    PRINT '                                FATURA DE EVENTO                                ';
    PRINT '=====';
    PRINT 'ID Evento:      ' + CAST(@ID_Evento AS VARCHAR(10));
    PRINT 'Data/Hora:      ' + CONVERT(VARCHAR(20), @Data_evento, 120);
    PRINT 'NIF Cliente:    ' + @NIF_Cliente;
    PRINT 'Local:          ' + @Nome_Local + ' (Custo Base: ' + CAST(@Custo_Local AS VARCHAR(15)) + '€)';
    PRINT '-----';
    PRINT '                                DETALHE DOS ITENS                                ';
    PRINT '-----';

    DECLARE cursor_detalhe_fatura CURSOR FOR
    SELECT
        A.ID_atividade,
        V.PrecoFinal
    FROM PROJETO_GESTAO_EVENTOS.Inclui I
    INNER JOIN PROJETO_GESTAO_EVENTOS.Atividade A ON I.ID_atividade_inclui = A.ID_atividade
    INNER JOIN PROJETO_GESTAO_EVENTOS.vw_PrecosAtividades V ON A.ID_atividade = V.ID_atividade
    WHERE I.ID_evento_inclui = @ID_Evento;

    OPEN cursor_detalhe_fatura;

    FETCH NEXT FROM cursor_detalhe_fatura INTO @ID_atividade, @Preco_Atividade;

    IF @@FETCH_STATUS <= 0
    BEGIN
        PRINT ' (Sem atividades extra adicionadas a este evento)';
    END

    WHILE @@FETCH_STATUS = 0
    BEGIN
        PRINT ' Item: Atividade Extra (ID: ' + CAST(@ID_atividade AS VARCHAR(10)) +
            ')-> Valor: ' + CAST(@Preco_Atividade AS VARCHAR(15)) + '€';

        FETCH NEXT FROM cursor_detalhe_fatura INTO @ID_atividade, @Preco_Atividade;
    END;

    CLOSE cursor_detalhe_fatura;
    DEALLOCATE cursor_detalhe_fatura;

    PRINT '-----';
    PRINT ' VALOR TOTAL A PAGAR: ' + CAST(@Custo_Total_Calculado AS VARCHAR(15)) + '€';
    PRINT '=====';
END;

```

Figura 6: Código da Stored procedure para imprimir a fatura do evento.

View

Dado o número de atividades disponíveis no sistema, tendo cada uma delas as suas sub-entidades com preços diferentes, foi necessária a criação de uma view que agregasse todas estas informações.

```
USE [p7g1]
GO
SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE VIEW [PROJETO_GESTAO_EVENTOS].[vw_PrecosAtividades] AS
SELECT
    A.ID_atividade,
    CASE
        -- Fotografia: preco_sessao * n_sesoes
        WHEN F.ID_atividade_fotografia IS NOT NULL THEN (F.preco_sessao * F.n_sesoes)

        -- Musica (Banda ou DJ): valor direto
        WHEN B.ID_atividade_banda IS NOT NULL THEN B.preco_banda
        WHEN D.ID_atividade_DJ IS NOT NULL THEN D.preco_DJ

        -- Animacao: preco_tema * n_horas * n_animadores
        WHEN AN.ID_atividade_animacao IS NOT NULL THEN
            (T.preco_tema * AN.n_horas_animacao * AN.n_animadores)

        -- Catering: preço do menu associado
        WHEN C.ID_atividade_catering IS NOT NULL THEN M.preco_menu

        ELSE 0
    END AS PrecoFinal
FROM PROJETO_GESTAO_EVENTOS.Atividade A
LEFT JOIN PROJETO_GESTAO_EVENTOS.Fotografia F ON A.ID_atividade = F.ID_atividade_fotografia
LEFT JOIN PROJETO_GESTAO_EVENTOS.Banda B ON A.ID_atividade = B.ID_atividade_banda
LEFT JOIN PROJETO_GESTAO_EVENTOS.DJ D ON A.ID_atividade = D.ID_atividade_DJ
LEFT JOIN PROJETO_GESTAO_EVENTOS.Animacao AN ON A.ID_atividade = AN.ID_atividade_animacao
LEFT JOIN PROJETO_GESTAO_EVENTOS.Tema T ON AN.ID_tipo_tema_animacao = T.ID_tipo_tema
LEFT JOIN PROJETO_GESTAO_EVENTOS.Catering C ON A.ID_atividade = C.ID_atividade_catering
LEFT JOIN PROJETO_GESTAO_EVENTOS.Menu M ON C.n_menu_catering = M.n_menu;
GO
```

Figura 7: Código da view para mostrar os preços das atividades ao utilizador.

UDF

Tendo por base a existência de várias atividades incluídas num dado evento foi necessária a criação de uma user defined function que permitisse o cálculo do valor total do evento tendo por base o valor das atividades e do espaço escolhido.

```
USE [p7g1]
GO
/***** Object: UserDefinedFunction [PROJETO_GESTAO_EVENTOS].[fn_CalcularCustoTotalEvento]    Script Da
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

ALTER FUNCTION [PROJETO_GESTAO_EVENTOS].[fn_CalcularCustoTotalEvento] (@ID_Evento INT)
RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @CustoLocal DECIMAL(10,2);
    DECLARE @CustoAtividades DECIMAL(10,2);
    DECLARE @Total DECIMAL(10,2);

    -- 1. Obter custo do local através da ligação Evento -> Local
    SELECT @CustoLocal = ISNULL(L.preco_local, 0)
    FROM PROJETO_GESTAO_EVENTOS.Evento E
    INNER JOIN PROJETO_GESTAO_EVENTOS.Local L ON E.Local_evento = L.ID_local
    WHERE E.ID_evento = @ID_Evento;

    -- 2. Somar custo das atividades extra (da tabela Inclui + Atividade)
    SELECT @CustoAtividades = ISNULL(SUM(V.PrecoFinal), 0)
    FROM PROJETO_GESTAO_EVENTOS.Inclui I
    INNER JOIN PROJETO_GESTAO_EVENTOS.vw_PrecosAtividades V ON I.ID_atividade_inclui = V.ID_atividade
    WHERE I.ID_evento_inclui = @ID_Evento;

    SET @Total = ISNULL(@CustoLocal, 0) + ISNULL(@CustoAtividades, 0);

    RETURN @Total;
END;
```

Figura 8: Código da UDF para calcular o custo total do evento.

Triggers

Os triggers, tendo sido criados com base nas nossas regras de negócio são scripts fundamentais para garantir o bom funcionamento do sistema, tendo no total, sido implementados 4.

Limite de participantes:

Previne casos de sobrelotação de espaços.

```
ALTER TRIGGER [PROJETO_GESTAO_EVENTOS].[TRG_capacidade_maxima]
ON [PROJETO_GESTAO_EVENTOS].[Evento]
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM inserted I
        INNER JOIN PROJETO_GESTAO_EVENTOS.[Local] L ON I.Local_evento = L.ID_local
        WHERE I.Participantes > L.Max_capacidade
    )
    BEGIN
        RAISERROR ('Limite de lotação. O número de participantes do evento ultrapassa a capacidade máxima do local escolhido.', 16, 1);
        ROLLBACK TRANSACTION;
    END;
END;
```

Figura 9: Código do trigger para prevenir a sobrelotação do espaço escolhido.

Prevenção de datas:

Garante que os eventos apenas são agendados para datas futuras.

```
ALTER TRIGGER [PROJETO_GESTAO_EVENTOS].[TRG_CheckDataFutura]
ON [PROJETO_GESTAO_EVENTOS].[Evento]
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (SELECT 1 FROM inserted I WHERE I.Data_hora_evento < GETDATE())
    BEGIN
        RAISERROR ('Não pode agendar eventos com data/hora já passadas.', 16, 1);
        ROLLBACK TRANSACTION;
    END
END;
```

Figura 10: Código do trigger para permitir agendar um evento apenas para datas futuras.

Prevenção de overbooking:

Limita a que só exista um evento por espaço durante o mesmo horário.

```
ALTER TRIGGER [PROJETO_GESTAO_EVENTOS].[TRG_prevenir_overbooking]
ON [PROJETO_GESTAO_EVENTOS].[Evento]
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM PROJETO_GESTAO_EVENTOS.Evento E
        INNER JOIN inserted I ON E.Local_evento = I.Local_evento
        WHERE E.ID_evento <> I.ID_evento
        AND E.Data_hora_evento < I.Data_hora_fin_evento
        AND E.Data_hora_fin_evento > I.Data_hora_evento
    )
    BEGIN
        RAISERROR ('Já existe um evento agendado para este local no mesmo intervalo de tempo.', 16, 1);
        ROLLBACK TRANSACTION;
    END
END;
```

Figura 11: Código do trigger para prevenir o overbooking.

Verificação de atividades:

Limita os eventos a que apenas tenham as atividades que lhes são permitidas.

```
ALTER TRIGGER [PROJETO_GESTAO_EVENTOS].[TRG_restricao_atividade_tipo_evento]
ON [PROJETO_GESTAO_EVENTOS].[Inclui]
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS(
        SELECT 1
        FROM inserted I
        INNER JOIN PROJETO_GESTAO_EVENTOS.Evento E ON I.ID_evento_inclui = E.ID_evento
        LEFT JOIN PROJETO_GESTAO_EVENTOS.Ten_atividade TA ON TA.Tipo_evento_tem_atividade = E.ID_tipo_evento
        AND I.ID_atividade_inclui = TA.Atividade_tem_atividade
        WHERE TA.Atividade_tem_atividade IS NULL
    )
    BEGIN
        RAISERROR ('Esta atividade não é permitida para o tipo de evento selecionado.', 16, 1);
        ROLLBACK TRANSACTION;
    END
END;
```

Figura 12: Código do trigger para verificar os tipos de atividades permitidas de acordo com o evento escolhido.

Índices

Sendo um sistema com vista a uso geral, certas tabelas têm maior predisposição a ter um maior número de registos, para essas tabelas foram desenhados índices, de forma a ter uma iteração mais eficaz das mesmas.

Evento:

Desenvolvidos índices para data e cliente que agendou o evento.

```
CREATE NONCLUSTERED INDEX [IX_Evento_Cliente_Local] ON [PROJETO_GESTAO_EVENTOS].[Evento]
(
    [Cliente_evento] ASC,
    [Local_evento] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
GO

CREATE NONCLUSTERED INDEX [IX_Evento_Data] ON [PROJETO_GESTAO_EVENTOS].[Evento]
(
    [Data_hora_evento] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
GO
```

Figura 13: Código dos índices para data e cliente na tabela Evento.

Cliente:

Criados índices para nome e email que agendou o evento.

```
CREATE NONCLUSTERED INDEX [IX_Cliente_Email] ON [PROJETO_GESTAO_EVENTOS].[Cliente]
(
    [Email] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
GO

CREATE NONCLUSTERED INDEX [IX_Cliente_Nome] ON [PROJETO_GESTAO_EVENTOS].[Cliente]
(
    [Nome_cliente] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
GO
```

Figura 14: Código para os índices para o nome e email na tabela Cliente.

Inclui:

Índice projetado para atividade.

```
CREATE NONCLUSTERED INDEX [IX_Inclui_Atividade] ON [PROJETO_GESTAO_EVENTOS].[Inclui]
(
    [ID_atividade_inclui] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
GO
```

Figura 15: Código do índice para as atividades da tabela Inclui.

Cursor

Como era possível observar no capítulo de Stored Procedures, durante o desenho do stored procedure que permite a emissão de faturas foi necessária a criação de um cursor para percorrer todas as atividades selecionadas para o evento.

```
DECLARE cursor_detalhe_fatura CURSOR FOR
SELECT
    A.ID_atividade,
    V.PrecoFinal
FROM PROJETO_GESTAO_EVENTOS.Inclui I
INNER JOIN PROJETO_GESTAO_EVENTOS.Atividade A ON I.ID_atividade_inclui = A.ID_atividade
INNER JOIN PROJETO_GESTAO_EVENTOS.vw_PrecosAtividades V ON A.ID_atividade = V.ID_atividade
WHERE I.ID_evento_inclui = @ID_Evento;

OPEN cursor_detalhe_fatura;

FETCH NEXT FROM cursor_detalhe_fatura INTO @ID_atividade, @Preco_Atividade;

IF @@FETCH_STATUS <> 0
BEGIN
    PRINT ' (Sem atividades extra adicionadas a este evento)';
END

WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT ' Item: Atividade Extra (ID: ' + CAST(@ID_atividade AS VARCHAR(10)) + |
        ') -> Valor: ' + CAST(@Preco_Atividade AS VARCHAR(15)) + '€';

    FETCH NEXT FROM cursor_detalhe_fatura INTO @ID_atividade, @Preco_Atividade;
END;

CLOSE cursor_detalhe_fatura;
DEALLOCATE cursor_detalhe_fatura;

PRINT '-----';
PRINT ' VALOR TOTAL A PAGAR: ' + CAST(@Custo_Total_Calculado AS VARCHAR(15)) + '€';
PRINT '=====';
```

Figura 16: Código do cursor para imprimir o nome e preço das atividades escolhidas.

Conclusão

Podemos concluir que os principais objetivos e as principais funcionalidades da aplicação para a gestão de eventos foram cumpridas. Permitindo assim uma gestão centralizada de eventos, de uma forma simplificada e de acesso direto para o utilizador.